

WARSZTAT PRAKTYCZNY: OPTYMALIZACJA BAZ DANYCH POSTGRESQL I ZARZĄDZANIE WSPÓŁBIEŻNOŚCIĄ

1. Przygotowanie Środowiska Bazo-Danowego

W ramach warsztatu wcielamy się w rolę Inżynierów Baza Danych (DBA / Database Performance Engineer) w dynamicznie rozwijającej się platformie e-commerce. System odnotowuje spadek wydajności w godzinach szczytu transakcyjnego. Celem warsztatu jest zdiagnozowanie problemów za pomocą wbudowanych narzędzi PostgreSQL oraz wdrożenie odpowiednich optymalizacji (strukturalnych, indeksowych oraz parametrycznych).

1.1. Skrypt Inicjalizacyjny Bazy Danych (DDL & DML)

Przed rozpoczęciem wykonywania zadań, należy wykonać poniższy skrypt SQL w środowisku PostgreSQL. Skrypt tworzy niezbędne tabele oraz generuje dane przy użyciu funkcji deterministycznych (setseed(0.42)).

```
-- Skrypt inicjalizacyjny dla warsztatu optymalizacyjnego PostgreSQL
-- Tworzenie bazy i tabel produkcyjnych z ziarnem losowości setseed(0.42)
```

```
DROP TABLE IF EXISTS order_items CASCADE;
DROP TABLE IF EXISTS orders CASCADE;
DROP TABLE IF EXISTS customers CASCADE;
DROP TABLE IF EXISTS products CASCADE;
DROP TABLE IF EXISTS index_write_test CASCADE;
DROP TABLE IF EXISTS planner_stats_demo CASCADE;
DROP TABLE IF EXISTS blocking_demo CASCADE;
```

```
SELECT setseed(0.42);
```

```
-- 1. Tabela Klienci (50 000 wierszy)
CREATE TABLE customers (
    customer_id BIGSERIAL PRIMARY KEY,
    customer_name TEXT NOT NULL,
    email TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```
INSERT INTO customers (customer_name, email, created_at)
SELECT
    'Klient_' || i,
    'user_' || i || '@example.com',
    now() - (random() * interval '3 years')
FROM generate_series(1, 50000) AS i;
```

```

-- 2. Tabela Zamówienia (200 000 wierszy)
CREATE TABLE orders (
    order_id BIGSERIAL PRIMARY KEY,
    customer_id BIGINT NOT NULL REFERENCES customers(customer_id),
    order_date DATE NOT NULL,
    status TEXT NOT NULL,
    shipping_region TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

INSERT INTO orders (customer_id, order_date, status, shipping_region, created_at)
SELECT
    1 + floor(random() * 50000)::bigint,
    CURRENT_DATE - floor(random() * 730)::int,
    CASE
        WHEN random() < 0.72 THEN 'completed'
        WHEN random() < 0.86 THEN 'shipped'
        WHEN random() < 0.94 THEN 'pending'
        WHEN random() < 0.98 THEN 'cancelled'
        ELSE 'returned'
    END,
    (ARRAY['North America', 'Europe', 'Asia Pacific', 'Latin America'])[1 + floor(random() *
4)::int],
    now() - (random() * interval '2 years')
FROM generate_series(1, 200000);

-- 3. Tabela Pozycje Zamówienia (500 000 wierszy)
CREATE TABLE order_items (
    item_id BIGSERIAL PRIMARY KEY,
    order_id BIGINT NOT NULL REFERENCES orders(order_id),
    product_id BIGINT NOT NULL,
    quantity INT NOT NULL,
    unit_price NUMERIC(10,2) NOT NULL
);

INSERT INTO order_items (order_id, product_id, quantity, unit_price)
SELECT
    1 + floor(random() * 200000)::bigint,
    1 + floor(random() * 10000)::bigint,
    1 + floor(random() * 10)::int,
    round((10 + random() * 490)::numeric, 2)
FROM generate_series(1, 500000);

-- 4. Tabela Produkty z danymi JSONB (10 000 wierszy)
CREATE TABLE products (
    product_id BIGSERIAL PRIMARY KEY,
    product_name TEXT NOT NULL,
    category TEXT NOT NULL,
    unit_price NUMERIC(10,2) NOT NULL,
    product_details JSONB NOT NULL
);

```

```

INSERT INTO products (product_name, category, unit_price, product_details)
SELECT
    'Produkt_' || i,
    (ARRAY['Electronics', 'Apparel', 'Home & Garden', 'Sports'])[1 + floor(random() * 4)::int],
    round((15 + random() * 985)::numeric, 2),
    jsonb_build_object(
        'brand', (ARRAY['Carved Rock Fitness', 'TechPro', 'EcoLife', 'PeakPerformance'])[1 +
        floor(random() * 4)::int],
        'color', (ARRAY['black', 'blue', 'red', 'green', 'white'])[1 + floor(random() * 5)::int],
        'weight_kg', round((0.1 + random() * 10)::numeric, 2)
    )
FROM generate_series(1, 10000) AS i;

-- Przeliczenie statystyk bazowych
ANALYZE customers;
ANALYZE orders;
ANALYZE order_items;
ANALYZE products;

```

2. Moduł 1: Analiza Planów Zapytań i Zaawansowane Indeksowanie B-Tree & GIN

Celem tego modułu jest opanowanie umiejętności czytania planów zapytań generowanych przez klauzulę EXPLAIN (ANALYZE, BUFFERS) oraz dobranie optymalnych struktur indeksowych (jednokolumnowych, złożonych, częściowych, pokrywających oraz GIN).

Teoria: Kluczowe Węzły Planera PostgreSQL

1. Seq Scan / Parallel Seq Scan: Odczyt całej tabeli blok po bloku od początku do końca. Praca może być dzielona na procesy pomocnicze (workers).
2. Bitmap Index Scan: Skanowanie indeksu i budowanie mapy bitowej adresów stron pamięci dyskowej (Heap Blocks).
3. Bitmap Heap Scan: Odczyt właściwych bloków tabeli na podstawie utworzonej mapy bitowej (optymalizuje odczyty dyskowe).
4. Index Scan / Index Only Scan: Bezpośredni odczyt z indeksu. Index Only Scan nie wymaga sięgania do tabeli głównej (Heap Fetches: 0), jeśli wskaźnik widoczności (Visibility Map) potwierdza aktualność stron.

Zadanie 1.1: Filtrowanie po dacie i podstawowy indeks B-Tree

Uruchom zapytanie baseline wyszukujące łączną wartość zamówień z ostatnich 30 dni:

```

-- Zapytanie bazowe (Baseline Query)
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    o.order_id,
    o.order_date,
    c.customer_name,

```

```

        SUM(oi.quantity * oi.unit_price) AS total_amount
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.order_date >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY o.order_id, o.order_date, c.customer_name
ORDER BY total_amount DESC
LIMIT 25;

```

Wycinek planu przed zindeksowaniem wykazuje skanowanie sekwencyjne na dużej tabeli zamówień:
 -> Parallel Seq Scan on orders o (cost=0.00..8689.83 rows=8852 width=20) (actual time=0.008..11.118 rows=7076 loops=3)
 Filter: (order_date >= (CURRENT_DATE - '30 days'::interval))
 Rows Removed by Filter: 159590
 Buffers: shared hit=3860 read=1184

Zadanie: Utwórz indeks B-Tree na kolumnie order_date tabeli orders, przeanalizuj zmianę planu zapytania.

```

-- Tworzenie indeksu B-Tree na dacie zamówienia
CREATE INDEX idx_orders_order_date .....

-- Ponowne wykonanie EXPLAIN (ANALYZE, BUFFERS)...

```

Wynik po dodaniu indeksu (uwaga wyniki mogą się od siebie różnić):
 -> Parallel Bitmap Heap Scan on orders o (cost=249.77..5453.09 rows=9104 width=20) (actual time=0.567..4.098 rows=10614 loops=2)
 Recheck Cond: (order_date >= (CURRENT_DATE - '30 days'::interval))
 -> Bitmap Index Scan on idx_orders_order_date (cost=0.00..244.30 rows=21850 width=0) (actual time=0.744..0.744 rows=21229 loops=1)
 Index Cond: (order_date >= (CURRENT_DATE - '30 days'::interval))
 Buffers: shared hit=3 read=22

Wniosek: Czas wykonania oraz koszt spadły, ponieważ PostgreSQL skanuje jedynie indeks `idx\_orders\_order\_date`, odrzucając zbędne strony dyskowe.

Zadanie 1.2: Indeks Złożony (Composite Index)

Analitycy dodali nowe kryterium biznesowe: interesują nas wyłącznie zamówienia o statusie 'completed' z ostatnich 30 dni.

```

EXPLAIN (ANALYZE, BUFFERS)
SELECT
    o.order_id, o.order_date, o.status, c.customer_name,
    SUM(oi.quantity * oi.unit_price) AS total_amount
FROM orders o
JOIN customers c ON c.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.order_date >= CURRENT_DATE - INTERVAL '30 days'
    AND o.status = 'completed'
GROUP BY o.order_id, o.order_date, o.status, c.customer_name
ORDER BY total_amount DESC LIMIT 25;

```

Przy użyciu indeksu jednokolumnowego `idx\_orders\_order\_date`, baza musi po pobraniu wierszy z indeksu dodatkowo filtrować wiersze pod kątem statusu. Utwórz indeks złożony (Composite Index):

```
DROP INDEX IF EXISTS idx_orders_order_date_status;
CREATE INDEX idx_orders_order_date_status ON orders (order_date, status);
ANALYZE orders;
```

Plan po dodaniu indeksu złożonego wykorzystuje warunek złożony w indeksie:

-> Bitmap Index Scan on idx_orders_order_date_status (cost=0.00..290.23 rows=15416 width=0) (actual time=0.554..0.554 rows=15453 loops=1)

Index Cond: ((order_date >= (CURRENT_DATE - '30 days'::interval)) AND (status = 'completed'::text))

Zadanie 1.3: Indeks Częściowy (Partial Index)

Jeżeli raporty systemowe w 95% przypadków odpytują wyłącznie o zamówienia zrealizowane ('completed'), tworzenie indeksu indeksującego wszystkie statusy (np. returned, cancelled) jest marnotrawstwem pamięci RAM.

Utwórz indeks częściowy z klauzulą WHERE:

```
DROP INDEX IF EXISTS idx_orders_recent_completed;
CREATE INDEX idx_orders_recent_completed .....
```

Zaobserwuj zmniejszenie liczby odczytywanych bloków indeksu z pamięci podręcznej (Buffers: shared read spada z 21 do 17 w węźle Index Scan). Indeks częściowy jest mniejszy i szybciej mieści się w buforach RAM.

Zadanie 1.4: Indeks Pokrywający (Covering Index) i Index Only Scan

Najwyższym poziomem optymalizacji odczytu jest stworzenie indeksu pokrywającego, który zawiera wszystkie kolumny wymagane przez zapytanie. Klauzula INCLUDE pozwala dołączyć kolumny niebędące kluczem wyszukiwania do liści drzewa B-Tree.

```
-- Baseline query bez JOIN
EXPLAIN (ANALYZE, BUFFERS)
SELECT order_id, customer_id, order_date, status
FROM orders
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days' AND status = 'completed'
ORDER BY order_date DESC LIMIT 100;

-- Tworzenie indeksu pokrywającego z klauzulą INCLUDE
CREATE INDEX idx_orders_covering_recent_completed
.....
ANALYZE orders;
```

Porównanie planów zapytań przed i po optymalizacji indeksem pokrywającym (**uzupełnij tabelę**):

Metryka Wydajnościowa	Przed Indeksem Pokrywającym	Po Indeksie Pokrywającym (Covering Index)
Typ węzła (Node Type)		
Estimated Cost		
Execution Time		
Heap Fetches (Odczyty z tabeli)		
Shared Buffers		

Zadanie 1.5: Indeksowanie Kolumn Półstrukturalnych JSONB (Indeks GIN)

Tabela `products` zawiera dane w formacie JSONB. Zapytanie szuka produktów o marce 'Carved Rock Fitness' i kolorze 'black' za pomocą operatora zawierania `@>`:

```
-- Zapytanie JSONB bez indeksu (powoduje Seq Scan)
EXPLAIN (ANALYZE, BUFFERS)
SELECT product_id, product_name, category, unit_price, product_details
FROM products
WHERE product_details @> '{"brand": "Carved Rock Fitness", "color": "black"}'
ORDER BY unit_price DESC LIMIT 25;

-- Tworzenie indeksu GIN (Generalized Inverted Index)
DROP INDEX IF EXISTS idx_products_details_gin;
CREATE INDEX idx_products_details_gin .....

-- Ponowne wykonanie EXPLAIN ANALYZE...
```

Wniosek: Indeks GIN umożliwia natychmiastowe wyszukiwanie wewnątrz zagnieżdżonych struktur JSONB bez konieczności pełnego skanowania tabeli.

3. Moduł 2: Ciemna Strona Indeksowania (Narzut na Operacje DML)

Początkujący projektanci baz danych często dodają indeks do każdej kolumny występującej w klauzulach WHERE. Każdy indeks w bazie danych PostgreSQL musi być jednak na bieżąco aktualizowany podczas wykonywania operacji `INSERT`, `UPDATE` i `DELETE`. Ten moduł demonstruje mierzalny koszt nadmiernego indeksowania.

Zadanie 2.1: Test porównawczy masowego wstawiania danych (100 000 wierszy)

Wykonujemy test w dwóch krokach na specjalnie przygotowanej tabeli `index\_write\_test`.

```
-- Krok 1: Przygotowanie tabeli czystej (Tylko PRIMARY KEY)
DROP TABLE IF EXISTS index_write_test;
CREATE TABLE index_write_test (
    order_id BIGSERIAL PRIMARY KEY,
    customer_id BIGINT NOT NULL,
    order_date DATE NOT NULL,
    status TEXT NOT NULL,
    shipping_region TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

SELECT setseed(0.42);

-- Wstawienie 100 000 wierszy (Baseline - 1 Indeks PK)
EXPLAIN (ANALYZE, BUFFERS)
INSERT INTO index_write_test (customer_id, order_date, status, shipping_region, created_at)
SELECT
    1 + floor(random() * 50000)::bigint,
    CURRENT_DATE - floor(random() * 730)::int,
    CASE
        WHEN random() < 0.72 THEN 'completed'
        WHEN random() < 0.86 THEN 'shipped'
        WHEN random() < 0.94 THEN 'pending'
        WHEN random() < 0.98 THEN 'cancelled'
        ELSE 'returned'
    END,
    (ARRAY['North America', 'Europe', 'Asia Pacific', 'Latin America'])[1 + floor(random() * 4)::int],
    now() - (random() * interval '2 years')
FROM generate_series(1, 100000);
```

Krok 2: Czyszczenie tabeli (`TRUNCATE`), dodanie 5 dodatkowych indeksów i ponowne wstawienie tej samej paczki 100 000 wierszy:

```
TRUNCATE TABLE index_write_test RESTART IDENTITY;

-- Dodanie dodatkowych indeksów
CREATE INDEX idx_write_test_order_date ON index_write_test (order_date);
```

```
CREATE INDEX idx_write_test_status ON index_write_test (status);
CREATE INDEX idx_write_test_region ON index_write_test (shipping_region);
CREATE INDEX idx_write_test_status_region ON index_write_test (status, shipping_region);
CREATE INDEX idx_write_test_created_at ON index_write_test (created_at);

-- Powtórzenie wstawiania 100 000 wierszy z setseed(0.42)...
```

Analiza porównawcza statystyk operacji wejścia/wyjścia (I/O Buffers)

(uzupełnij tabelę):

Metryka Statystyk Bufora (Buffers)	Test A (Tylko 1 PRIMARY KEY)	Test B (PK + 5 Dodatkowych Indeksów)	Wzrost / Narzut
shared hit (odczyty z pamięci)			
dirtied (brudne strony w RAM)			
written (zapisy na dysk)			

4. Moduł 3: Przepisywanie Zapytań bez Dodawania Indeksów

Dodawanie indeksów nie jest jedynym sposobem optymalizacji. Często wąskim gardłem jest zła struktura samego zapytania SQL. W tym module uczymy się refaktoryzacji zapytań zawierających podzapytania korelacyjne na optymalne złączenia `JOIN` z grupowaniem `GROUP BY`.

Zadanie 3.1: Optymalizacja podzapytania korelacyjnego w klauzuli SELECT

Przeanalizuj poniższe zapytanie, które dla każdego zamówienia klienta z przedziału 1000-1100 wykonuje podzapytanie liczące sumę pozycji zamówienia:

```
-- Zapytanie nieoptymalne (z podzapytaniem korelacyjnym)
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    o.order_id,
    o.order_date,
    (
        SELECT SUM(oi.quantity * oi.unit_price)
        FROM order_items oi
        WHERE oi.order_id = o.order_id
    ) AS total_amount
FROM orders o
WHERE o.customer_id BETWEEN 1000 AND 1100
    AND o.order_date >= CURRENT_DATE - INTERVAL '180 days'
ORDER BY total_amount DESC
LIMIT 25;
```

W planie zapytania widoczny jest węzeł `SubPlan`, który wymusza iteracyjne wykonywanie podzapytania dla każdego pasującego wiersza z tabeli `orders`.

Zadanie: Przepisz zapytanie, zastępując podzapytanie korelacyjne jawnym złączeniem `JOIN order\_items` oraz klauzulą `GROUP BY`.

```
-- Zapytanie zrefaktoryzowane (Jawne JOIN i GROUP BY)
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    .....
FROM .....
```

Wynik refaktoryzacji: PostgreSQL zastępuje wolne podzapytanie efektywnym algorytmem złączenia (Hash Join / Merge Join), drastycznie redukując czas wykonania zapytania bez konieczności tworzenia jakichkolwiek nowych indeksów na dysku!

5. Moduł 4: Statystyki Planisty Zapytań i Zjawisko Data Drift

Optymalizator PostgreSQL opiera swoje decyzje (wybór między Seq Scan a Index Scan) na statystykach zgromadzonych w widoku `pg\_statistic` (odświeżanych przez proces ANALYZE/AUTOVACUUM). Gdy rozkład danych ulega gwałtownej zmianie (Data Drift), nieaktualne statystyki mogą doprowadzić planistę do wyboru katastrofalnie nieefektywnego planu.

Zadanie 4.1: Symulacja Data Drift na tabeli planner_stats_demo

Wykonaj poniższy kod tworzący zbiór testowy 300 000 wierszy, gdzie status 'completed' stanowi 95%, a 'cancelled' 5%:

```
DROP TABLE IF EXISTS planner_stats_demo;
CREATE TABLE planner_stats_demo (
    order_id BIGSERIAL PRIMARY KEY,
    status TEXT NOT NULL,
    order_date DATE NOT NULL,
    total_amount NUMERIC(10,2) NOT NULL
);

SELECT setseed(0.42);

INSERT INTO planner_stats_demo (status, order_date, total_amount)
SELECT
    CASE WHEN random() < 0.95 THEN 'completed' ELSE 'cancelled' END,
    CURRENT_DATE - floor(random() * 365)::int,
    round((25 + random() * 975)::numeric, 2)
FROM generate_series(1, 300000);

CREATE INDEX idx_planner_stats_demo_status ON planner_stats_demo (status);
ANALYZE planner_stats_demo;
```

Krok A: Zapytanie przy aktualnych statystykach dla statusu 'cancelled' (rzadki status - 5%):

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) FROM planner_stats_demo WHERE status = 'cancelled';
-- Planista poprawnie wybiera: Index Only Scan (cost=0.42..322.07 rows=15180 actual
time=0.016..0.456 ms)
```

Krok B: Symulacja wygenerowania olbrzymiej ilości anulowań (UPDATE 210 000 wierszy) BEZ odświeżania statystyk:

```
-- Zmiana rozkładu danych: 'cancelled' stanowi teraz 70% wierszy!
UPDATE planner_stats_demo SET status = 'cancelled' WHERE order_id <= 210000;

-- Wykonanie zapytania PRZED odświeżeniem statystyk (Baza myśli, że to nadal 5%)
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) FROM planner_stats_demo WHERE status = 'cancelled';
```

Obserwacja błędu planisty:

Planista szacuje, że zapytanie zwróci 25 805 wierszy (rows=25805), podczas gdy faktycznie zwraca 214 516 wierszy.

W konsekwencji wybiera `Index Only Scan`, co powoduje drastyczny spadek wydajności (czas skoczył z 0.45 ms do 16.5 ms).

Krok C: Naprawa statystyk komendą ANALYZE:

```
-- Odświeżenie statystyk planisty
ANALYZE planner_stats_demo;

-- Ponowne zapytanie po ANALYZE
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) FROM planner_stats_demo WHERE status = 'cancelled';
```

Wynik po ANALYZE:

Planista aktualizuje szacunki (rows=89346) i porzuca indeks na rzecz `Parallel Seq Scan` (actual time=0.208..4.093 ms rows=71505 loops=3).

Skanowanie sekwencyjne dla 70% wierszy jest ponad 4x szybsze niż skanowanie indeksu!

6. Moduł 5: Zarządzanie Współbieżnością i Diagnostowanie Blokad Transakcji

W środowiskach wielodostępnych nieodpowiednio zarządzane transakcje mogą prowadzić do blokowania zasobów (Locking) i zawieszania aplikacji. W tym module przeprowadzamy symulację blokady w osobnych sesjach oraz uczą się diagnozować podmiot blokujący za pomocą widoków systemowych.

Zadanie 5.1: Symulacja Blokady Transakcyjnej (Wykonywana w 3 sesjach psql)

Przygotuj tabelę testową `blocking\_demo`:

```
DROP TABLE IF EXISTS blocking_demo;
CREATE TABLE blocking_demo (
    order_id BIGSERIAL PRIMARY KEY,
    status TEXT NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

INSERT INTO blocking_demo (status) VALUES ('pending');
ANALYZE blocking_demo;
```

Krok / Sesja	Polecenie SQL do wykonania	Zachowanie i Obserwacja
Krok 1 (Sesja 1)	BEGIN; UPDATE blocking_demo SET status = 'processing', updated_at = now() WHERE order_id = 1; -- (Nie wykonuj COMMIT ani ROLLBACK!)	Transakcja otwiera blokadę wyłączną (Exclusive Lock) na wierszu order_id = 1. Sesja czeka.
Krok 2 (Sesja 2)	UPDATE blocking_demo SET status = 'completed', updated_at = now() WHERE order_id = 1;	Zapytanie w Sesji 2 ZAWIESZA SIĘ (zawieszenie terminala/ksera), czekając na zwolnienie blokady przez Sesję 1.
Krok 3 (Sesja 3 - Diagnosta)	Uruchomienie zapytania diagnostycznego w pg_stat_activity i pg_blocking_pids (patrz skrypt poniżej).	Identyfikacja identyfikatora procesu (PID) bloku blokującego i blokowanego.
Krok 4 (Sesja 1)	COMMIT;	Zwolnienie blokady. Sesja 2 natychmiast odblokowuje się i kończy sukcesem.

Skrypt Diagnostyczny dla DBA (Sesja 3)

Poniższe zapytanie SQL pozwala administratorowi natychmiast wykryć, który proces (blocking_pid) blokuje inną transakcję (blocked_pid):

```
-- Zapytanie diagnostyczne: Relacja Blocker / Blockee w PostgreSQL
SELECT
    blocked.pid AS blocked_pid,
    blocked.query AS blocked_query,
    blocking.pid AS blocking_pid,
    blocking.state AS blocking_state,
```

```
    now() - blocking.xact_start AS blocking_transaction_age,  
    blocking.query AS blocking_query  
FROM pg_stat_activity blocked  
JOIN pg_stat_activity blocking  
    ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))  
WHERE blocked.datname = current_database();
```